WORKING WITH TERRAFORM MODULES AND DEPENDENCIES

WHAT IS TERRAFORM MODULE

1. A Terraform Module is simply a reusable collection of Terraform code.
2. Think of a module as a template or blueprint that contains resources you want to create repeatedly.

WHY MODULES ARE USED

1. Reusability
2. Organization
3. Avoid Repetition
4. No copy-paste. Just call the module.
5. Better collaboration

STRUCTURE OF A TERRAFORM MODULE

terraform-project/

```
│
├── main.tf
├── outputs.tf
│
└── modules/
    ├── vpc/
    │   ├── main.tf
    │   ├── variables.tf
    │   └── outputs.tf
    │
    └── ec2/
        ├── main.tf
        ├── variables.tf
        ├── outputs.tf
        └── user-data.sh
```

CREATION OF VPC & EC2 USING MODULE

VPC FILES

1.main.tf

```
resource "aws_vpc" "my_vpc" {
  cidr_block = var.vpc_cidr
  tags = {
    Name = "my-vpc"
  }
}

resource "aws_subnet" "public_subnet" {
  vpc_id = aws_vpc.my_vpc.id
  cidr_block = var.public_subnet_cidr
  availability_zone = var.public_az
  map_public_ip_on_launch = true
  tags = {
    Name = "public-subnet"
  }
}

resource "aws_subnet" "private_subnet" {
  vpc_id = aws_vpc.my_vpc.id
  cidr_block = var.private_subnet_cidr
  availability_zone = var.private_az
  tags = {
    Name = "private-subnet"
  }
}

resource "aws_internet_gateway" "IGW" {
  vpc_id = aws_vpc.my_vpc.id
  tags = {
    Name = "IGW"
  }
}

resource "aws_eip" "nat_eip" {
  domain = "vpc"
  tags = {
```

```
    Name = "nat-eip"
  }
}

resource "aws_nat_gateway" "nat_gateway" {
  subnet_id = aws_subnet.public_subnet.id
  allocation_id = aws_eip.nat_eip.id
  tags = {
    Name = "nat-gateway"
  }
}

resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.my_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.IGW.id
  }
  tags = {
    Name = "public-rt"
  }
}

resource "aws_route_table_association" "public_rt_assoc" {
  subnet_id = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_route_table" "private_rt" {
  vpc_id = aws_vpc.my_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat_gateway.id
  }
  tags = {
    Name = "private-rt"
  }
}

resource "aws_route_table_association" "private_rt_assoc" {
```

```
  subnet_id = aws_subnet.private_subnet.id
  route_table_id = aws_route_table.private_rt.id
}

resource "aws_security_group" "sg" {
  name = var.sg_name
  description = "my-security-group"
  vpc_id = aws_vpc.my_vpc.id

  ingress {
    from_port = 80
    to_port = 80
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 22
    to_port = 22
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port = 0
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = {
    Name = var.sg_name
  }
}
```

2.var.tf

```
variable "vpc_cidr" {}
variable "public_subnet_cidr" {}
variable "private_subnet_cidr" {}
variable "sg_name" {}
variable "public_az" {}
```

```
variable "private_az" {}
```

3.output.tf

```
output "sg_id" {
   value = aws_security_group.sg.id
}
output "public_subnet_id" {
   value = aws_subnet.public_subnet.id
}
output "private_subnet_id" {
   value = aws_subnet.private_subnet.id
}
output "vpc_id" {
   value = aws_vpc.my_vpc.id
}
```

EC2 FILES

1.main.tf

```
resource "aws_instance" "public_instance" {
  ami = var.ami
  instance_type = var.instance_type
  key_name = var.key_name
  subnet_id = var.public_subnet
  vpc_security_group_ids = [var.sg_id]
  associate_public_ip_address = true
  user_data = file("/root/terraform-b30/module/ec2/user-data.sh")
  tags = {
    Name = "public-instance"
  }
}

resource "aws_instance" "private_instance" {
  ami = var.ami
  instance_type = var.instance_type
  key_name = var.key_name
  subnet_id = var.private_subnet
  vpc_security_group_ids = [var.sg_id]
  user_data = file("/root/terraform-b30/module/ec2/user-data.sh")
  tags = {
```

```
  Name = "private-instance"
 }
}
```

2.var.tf

```
variable "ami" {}
variable "instance_type" {}
variable "key_name" {}
variable "public_subnet" {}
variable "private_subnet" {}
variable "sg_id" {
 description = "Security group ID"
 type        = string
}
```

3.output.tf

```
output "public_ip" {
  value = aws_instance.public_instance.public_ip
}
```

4.user-data.sh

```
#!/bin/bash
sudo apt update -y
sudo apt install apache2 -y
sudo systemctl start apache2
sudo systemctl enable apache2
sudo echo "<h1> Welcome to terraform </h1>" | tee /var/www/html/index.html
sudo systemctl restart apache2
```

MAIN FILES

1.main.tf

```
module "vpc" {
 source = "./module/vpc"
 vpc_cidr = "10.0.0.0/16"
 public_subnet_cidr = "10.0.0.0/24"
 private_subnet_cidr = "10.0.1.0/24"
 sg_name = "my-security-group-vpc"
 public_az = "ap-south-1a"
 private_az = "ap-south-1b"
```

```
}

module "ec2" {
  source = "./module/ec2"
  ami    = "ami-07a00cf47dbbc844c"
  instance_type = "t3.micro"
  key_name = "ubuntu"
  public_subnet = module.vpc.public_subnet_id
  private_subnet = module.vpc.private_subnet_id
  sg_id = module.vpc.sg_id
}
```

2.output.tf

```
output "vpc_id" {
  value = module.vpc.vpc_id
}
output "public_ip" {
  value = module.ec2.public_ip
}
```

3.provider.tf

```
provider "aws" {
    region = "ap-south-1"
    profile = "config"
}
```

1. git add .
2. git commit -m “module”
3. git push

ON UBUNTU-LOCAL

1. git clone https://github.com/mayurpandit25/terraform-b30.git
2. cd terraform-b30
3. terraform init
4. terraform apply –auto-approve
5. terraform destroy –auto-approve

TASK:

1. create a module for vpc / alb / asg
